

ANURAG NAVIN BHANDARY

+91 8169370520 | bhandaryanurag@gmail.com | anuragbhandary.github.io | linkedin.com/in/bhandary-anurag | github.com/AnuragBhandary | Mumbai, India

SUMMARY

Data engineer and analyst with a year on the founding team of an early-stage AI sports startup where game events moved through **Kafka** into live sports-statistics delivery. Works in **SQL**, **Python**, **Power BI**, and **Excel** across **PostgreSQL**, **MySQL**, **dbt**, **Airflow**, and **Kafka**, taking data from raw source tables to modeled, tested, and reconciled marts and the dashboards built on them.

TECHNICAL SKILLS

Languages: **SQL** (window functions, CTEs, query tuning), **Python** (pandas, SQLAlchemy), **Java** (core, DSA)

Databases: **PostgreSQL** (partitioning, indexing, materialized views), **MySQL 8**, **Redis**

BI and Analytics: **Power BI** (DAX, Power Query, row-level security), **Excel** (PivotTables, XLOOKUP, Power Query), Metabase

Data Engineering: **dbt**, **Apache Airflow**, **Apache Kafka**, **Debezium** (CDC), dimensional modeling, SCD Type 2, data quality testing

Cloud and Tools: **Docker**, **Docker Compose**, **AWS S3**, **Git**

EXPERIENCE

Software Developer (Founding Team) | **Fayble, LLC** | **Remote** | **Jan 2026 - Jun 2026**

Software Development Intern | **Fayble, LLC** | **Remote** | **Jul 2025 - Dec 2025**

AI sports-simulation platform

- Engineered the **audio pipeline** for a real-time AI sports-commentary system, generating dual-commentator speech with **ElevenLabs** and stitching per-play clips into continuous broadcast output.
- Measured text-generation latency for **Gemini** and **Groq**, found Groq roughly **10x faster**, and used the result to restructure generation around one emotion per play, cutting audio-generation time about **threefold**.
- Automated archery match production with **ffmpeg**, joining AI commentary, multi-channel audio, and **structured per-play metadata** into finished match videos.
- Integrated **Apache Kafka** into the platform's animation flow and sports-statistics delivery, replaying **19 games** (about **1,274 event messages**) through an ordered ingestion path that rejected duplicates, serving live game state over **FastAPI** and **WebSockets**.

PROJECTS

E-Commerce Analytics Warehouse | *MySQL, PostgreSQL, Python, dbt, Airflow, Power BI, Excel, Docker*

- Replayed the public **Olist** dataset (about **100,000 orders**) into **MySQL** as a live operational source, and modeled a **PostgreSQL star schema** in **dbt**: order, payment, and review facts over customer, product, seller, and date dimensions.
- Scheduled **idempotent incremental loads** in **Airflow** with **SCD Type 2** customer snapshots, enforced by **85 dbt tests** and row-level **reconciliation** against MySQL.
- Built a 4-page **Power BI** report on the star schema with **20+ DAX measures** (YoY and MoM time intelligence, cohort retention, on-time delivery rate), drill-through by seller, and **row-level security** by state.
- Delivered a monthly seller-performance **Excel** workbook fed by **Power Query** from the marts, with **PivotTables**, **XLOOKUP**, and conditional formatting, reconciled to the Power BI totals.

Change Data Capture Pipeline: MySQL to PostgreSQL | *MySQL, Debezium, Apache Kafka, Python, PostgreSQL, Docker*

- Streamed row-level changes from the **MySQL binlog** through **Debezium** and **Kafka** into **PostgreSQL**, applying inserts, updates, and deletes as **idempotent upserts** keyed on primary key and source position.
- Held **p95 replication lag at 141 ms** while replaying **1 million change events**, with zero lost or duplicated rows confirmed by per-table **checksums** against the source.
- Propagated added columns without downtime, and resumed a consumer killed mid-batch from committed **Kafka offsets** with no drift between source and target.

PostgreSQL Query Performance on NYC Taxi Data | *PostgreSQL, MySQL 8, SQL, Python*

- Loaded about **38 million** 2023 NYC TLC yellow-taxi trips into **PostgreSQL** with **COPY**, **range-partitioned by month**, and wrote 12 analytical queries using **window functions**, **CTEs**, and percentile aggregates.
- Cut the suite's median query from **488 ms to 19 ms** with **partition pruning**, **BRIN** and covering indexes, and **materialized views**, verifying identical results and every plan with **EXPLAIN ANALYZE**; compared against **MySQL 8**.

EDUCATION

M.S. Computer Science | **University of Texas at Arlington** | **Aug 2023 - May 2025**

GPA 3.42/4.0 | Focus: Artificial Intelligence and Software Engineering | Arlington, TX

B.E. Electronics Engineering | **University of Mumbai** | **Aug 2019 - May 2023**

GPA 8.36/10 | Mumbai, India

Competitive Coding: LeetCode 300+ problems solved in Python and Java across data structures and algorithms |

leetcode.com/anuragb2901